

Estudio comparativo de heurísticas de construcción de un BVH

Trazado de rayos en CPU

CS3014 – Estructura de Datos Avanzados (2026-1)

Autor 1, Autor 2, Autor 3

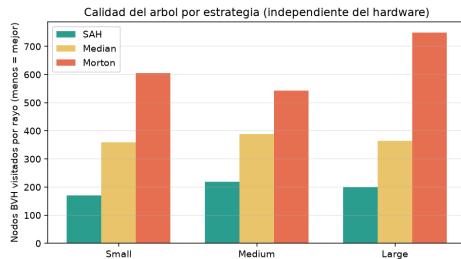
El problema

- ▶ Trazado de rayos: por cada píxel se emite ≥ 1 rayo; con sombras y reflejos, varios.
- ▶ Fuerza bruta: $O(P \cdot N)$ por frame para P píxeles y N triángulos $\Rightarrow$ injugable con decenas de miles de triángulos.
- ▶ **Solución**: una estructura que descarte grandes grupos de triángulos con pruebas baratas.

Objetivo del proyecto: **comparar estrategias de construcción** del BVH y medir su calidad frente a la referencia industrial (Intel Embree).

BVH: la idea

- ▶ Árbol binario de cajas alineadas a los ejes (AABB).
- ▶ La raíz envuelve toda la escena; cada nodo parte sus triángulos en dos hijos; las hojas guardan pocos triángulos.
- ▶ Un rayo prueba las cajas baratas primero y solo desciende por las que golpea $\Rightarrow O(N) \rightarrow \sim O(\log N)$.
- ▶ Memoria $O(N)$.



Calidad = nodos visitados por rayo (menos = mejor árbol).

Heurística SAH (área de superficie)

Un rayo es más probable de entrar en una caja grande. El costo esperado de un corte se estima como:

$$C_{\text{split}} = 1 + \frac{A_L n_L + A_R n_R}{A_{\text{padre}}}$$

- ▶ Se elige el corte de menor costo, comparándolo con no partir (hoja).
- ▶ Para no evaluar todos los cortes: *binning* (12 cubos por eje) $\Rightarrow$ build $O(N \log N)$, cerca del óptimo.

Recorrido: iterativo con pila fija; rayo-AABB por *slabs*; rayo-triángulo por Möller-Trumbore.

Tres estrategias de construcción

- ▶ **SAH (binned)**: corte óptimo por costo. **Mejor calidad**, build algo más caro.
- ▶ **Median**: parte por la mediana de centroides (eje más largo). Sencilla, calidad intermedia.
- ▶ **Morton/LBVH**: ordena todo por código de Morton (Z-order) y parte por índice medio. **Build más rápido**, calidad menor.

Las tres son seleccionables en vivo desde el menú de la aplicación; al cambiar, se reconstruye el árbol y se actualizan las métricas.

Setup experimental

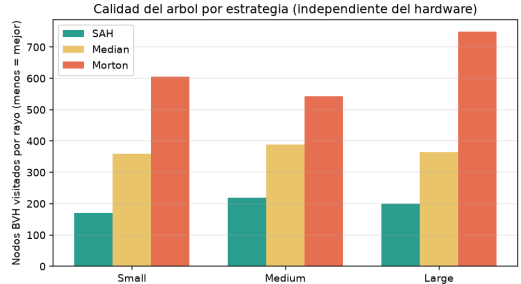
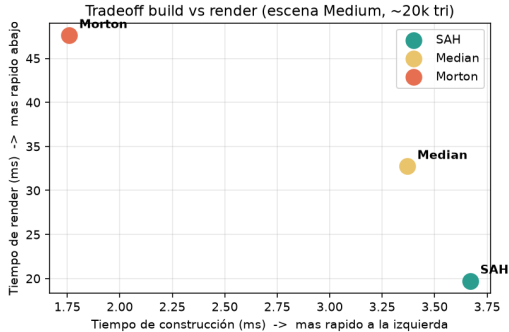
- ▶ Escena: ciudad procedural (fachadas con ventanas) en 3 tamaños: *Small/Medium/Large* (~16k–33k triángulos).
- ▶ Sobre los **mismos rayos primarios** medimos:
 - ▶ **build**: tiempo de construir el árbol.
 - ▶ **nodos/ray**: calidad del árbol (independiente del hardware).
 - ▶ **traversal**: intersección pura, single-thread, sin sombreado.
- ▶ Referencia externa: **Intel Embree 4** sobre la misma geometría (`rtcIntersect1`).
- ▶ Plataforma Apple Silicon (ARM64/NEON), `-O2 -march=native`.
- ▶ Reproducible: `--bench` genera `benchmark.csv`.

Resultados – escena Medium ($\sim 19,750$ tri)

Estrategia	build (ms)	render (ms)	traversal (ms)	nodos/ray
SAH	3.67	19.7	27.8	218
Median	3.37	32.7	45.5	388
Morton	1.76	47.6	64.3	542
Embree	2.31	—	9.96	—

SAH: mejor árbol y mejor render. Morton: build más rápido pero peor árbol. Embree: traversal $\sim 3\times$ más rápido.

Resultados – tradeoff y calidad

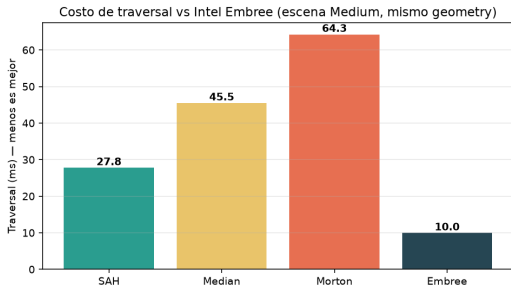


Nodos/ray por estrategia.

Build vs. render.

Morton intercambia calidad por velocidad de build; SAH domina en calidad.

Resultados – vs. Intel Embree



Traversal (ms), mismo geometry.

- ▶ Embree recorre $\sim 3\times$ más rápido que nuestra SAH.
- ▶ **No** por la calidad del árbol (nuestra SAH es competitiva, ~ 218 nodos/ray).
- ▶ Sí por la **ingeniería del kernel**: SIMD (NEON), nodos de 4/8 vías (MBVH) y layout de caché.

- ▶ **Entre nuestras heurísticas:** se confirma el tradeoff build–calidad. SAH es el ganador global en tiempo de render.
- ▶ **Frente a Embree:** la brecha está en la **implementación** del recorrido (SIMD), no en la **estructura de datos**.
- ▶ **Lección:** nuestra BVH/SAH alcanza calidad comparable; cerrar la brecha requiere ingeniería de kernels, no un mejor árbol.

Limitaciones y trabajo futuro

- ▶ Recorrido **SIMD/NEON**: procesar varios rayos/hijos por instrucción.
- ▶ **MBVH**: nodos de 4 vías $\Rightarrow$ menos nodos visitados.
- ▶ **Layout SoA** y nodos cuantizados para mejor uso de caché.
- ▶ Construcción paralela y comparar con **SBVH** (espacial) para geometría que se intersecta.

Conclusiones y contribuciones

- ▶ Ray tracer CPU con BVH + **benchmark reproducible** (`--bench`).
- ▶ Comparación de 3 heurísticas (SAH/Median/Morton) y de la referencia Embree sobre la misma geometría.
- ▶ Hallazgo: SAH da el mejor árbol; Morton prioriza build; Embree gana en traversal por SIMD.

Contribuciones: A1 – BVH y heurísticas; A2 – escena y renderer; A3 – métricas, benchmark y Embree.

Repositorio, datos y gráficas en el GitHub del proyecto; video demo adjunto.

¡Gracias!

Preguntas y respuestas